

RELIOMQ DELIVERY MODES

คู่มือเลือกและตั้งค่า

Fast, Group และ Durable Mode

อธิบายจากภาพการทำงานจริง ไปจนถึง fsync, DeliveryAck, checkpoint, power loss และการตั้งค่าทุกพารามิเตอร์

สำหรับ reliomq 0.6.2

ฉบับปรับภาษาให้อ่านง่าย พร้อมตัวอย่าง config ใช้งานจริง

วิธีอ่านคู่มือนี้

ส่วนที่ 1 ช่วยให้เลือกโหมดและตั้งค่าได้เร็ว ส่วนที่ 2 อธิบายกลไกบน disk และผลเมื่อ process crash หรือไฟดับ หากต้องตัดสินใจทำงานให้อ่าน 1.1 ถึง 1.5 ก่อน แล้วค่อยใช้ส่วนที่ 2 ตรวจสอบเหตุผลทางเทคนิค

1. Mode Overview

1.1 Quick comparison

1.2 FastMode

1.3 GroupMode

1.4 DurableMode

1.5 Recommended selection

2. Deep Technical Detail

2.1 Storage architecture

2.2 write, flush, fsync

2.3 ACK model

2.4 checkpoint.json

2.5 FastMode in detail

2.6 GroupMode in detail

2.7 DurableMode in detail

2.8 Files by mode

2.9 Crash and restart

2.10 Storage wear

2.11 Two durability windows

2.12 End-user wording

2.13 Glossary

คำว่า durable ในเอกสารนี้: ข้อมูลผ่านจุด synchronization ที่ไลบรารีกำหนดแล้ว แต่ความสามารถรอดจากไฟดับยังขึ้นกับ filesystem, mount option, storage controller และ hardware ด้วย

1. Mode Overview

reliomq มี 3 โหมดเพื่อแลกเปลี่ยนระหว่างความเร็ว อายุ storage และความทนทานต่อ crash วิธีแยกโหมดที่ง่ายที่สุดคือถาม 2 ข้อ:

1. payload ถูกทำให้ durable เมื่อใด?
2. สถานะที่บอกว่า DeliveryAck แล้ว ถูกทำให้ durable เมื่อใด?

write ไม่เท่ากับ fsync: การเขียนไฟล์อาจจบเพียงใน page cache ของระบบปฏิบัติการ ส่วน **fsync()** คือขอบเขตที่ขอให้ระบบผลักข้อมูลไปยัง stable storage และรอผล

1.1 Quick comparison

Mode	Payload ปกติ	ACK state	เมื่อไฟดับ	เหมาะกับ
Fast ค่าเริ่มต้น	เริ่มใน RAM; ลง disk เมื่อ spill/recovery	RAM-only ไม่ต้อง checkpoint; disk-backed ทำแบบ durable	รายการล่าสุดที่ยังอยู่เฉพาะ RAM อาจหาย	telemetry ถี่, ลด disk write
Group	append ทุกข้อความ แล้ว fsync เป็นชุด	เลื่อน cursor ใน RAM แล้ว checkpoint เป็นชุด	payload ที่ยังไม่ sync อาจหาย; ACK ล่าสุดอาจ replay	สมดุล reliability/throughput
Durable	append + fsync ต่อข้อความ	checkpoint ต่อ completion	ป้องกัน accepted-message loss ได้มากที่สุด	alarm, transaction, critical event

จำสั้น ๆ: Fast = RAM ก่อน | Group = เขียนก่อน sync ที่หลัง | Durable = เขียนและ sync ทันที

1.2 FastMode - speed and low storage wear

FastMode รับข้อความเข้าคิว RAM ที่มีขอบเขต แล้วส่งผ่าน MQTT หากได้รับ DeliveryAck ก่อนเกิด spill ข้อความนั้นจะไม่ถูก append ลง Outbox เลย จึงลด disk I/O ได้มากที่สุด



การตั้งค่าทุกพารามิเตอร์

พารามิเตอร์	ค่าเริ่มต้น / ช่วงค่า	ผลเมื่อปรับ
ram_max_messages	10,000 จำนวนเต็ม > 0	hard limit จำนวนข้อความที่ RAM เป็นเจ้าของ ปิดไม่ได้ หากเต็มและ spill ไม่สำเร็จ publish อาจเกิด FastQueueFullError
ram_max_bytes	33,554,432 32 MiB, จำนวนเต็ม > 0	hard limit ของ canonical envelope bytes ไม่ใช่ RAM ทั้ง process; object overhead และ buffer ชั่วคราวอยู่นอกค่านี
high_watermark	0.75 (0, 1] หรือ None	เริ่ม spill เมื่อจำนวนข้อความหรือ bytes ถึงสัดส่วนนี้ของ hard limit ใช้ None เพื่อปิดเฉพาะ proactive pressure trigger
max_ram_age	5.0 s > 0 หรือ None	spill ข้อความเก่าสุดเมื่ออยู่ใน RAM นานถึงกำหนด ใช้ None เพื่อไม่ spill เพราะอายุ
disconnect_grace	3.0 s >= 0 หรือ None	ยอมให้ MQTT disconnected ต่อเนื่องกี่วินาทีก่อน spill; 0 = spill ทันที, None = ปิด trigger จาก disconnect

พารามิเตอร์	ค่าเริ่มต้น / ช่วงค่า	ผลเมื่อปรับ
<code>spill_batch_messages</code>	1,000 จำนวนเต็ม > 0	จำนวนข้อความสูงสุดต่อ batch spill ปิดไม่ได้; ลดเพื่อลดเวลาที่ lock/เขียนชุดหนึ่ง เพิ่มเพื่อรวมงานมากขึ้น
<code>spill_batch_bytes</code>	4,194,304 4 MiB, จำนวนเต็ม > 0	เป้าหมาย serialized bytes ต่อ batch; record ที่ใหญ่กว่าค่านี้อาจ spill ได้ยวถ้ายังไม่เกิน <code>ram_max_bytes</code>

ตัวอย่างคำแนะนำ

```
from reliomq import FastMode

mode = FastMode(
    ram_max_messages=10_000,
    ram_max_bytes=32 * 1024 * 1024,
    high_watermark=0.75,
    max_ram_age=5.0,
    disconnect_grace=None,      # ไม่ spill เพราะ offline อย่างเดียว
    spill_batch_messages=1_000,
    spill_batch_bytes=4 * 1024 * 1024,
)
```

None ไม่ได้ปิด spill ทั้งหมด: hard capacity, delivery failure/timeout ขณะ connection พร้อม, clean shutdown และ trigger อื่นที่ยังเปิดอยู่ยังทำงาน

1.3 GroupMode - balanced mode

GroupMode append ทุกข้อความลง segment ทันทันที แต่เลื่อน `fsync` ไปทำเป็นชุด และแยกการ batch ของ payload ออกจากการ checkpoint ACK จึงควบคุมได้ทั้งช่วงเสี่ยงข้อมูลใหม่หายและช่วงเสี่ยงข้อมูลเก่า replay

Data window

append -> page cache -> fsync
ก่อน fsync: payload ล่าสุดอาจหายเมื่อไฟดับ

ACK window

DeliveryAck -> RAM cursor -> checkpoint
ก่อน checkpoint: ข้อความที่ส่งแล้วอาจ replay

การตั้งค่าทุกพารามิเตอร์

พารามิเตอร์	ค่าเริ่มต้น / ช่วงค่า	ผลเมื่อปรับ
<code>sync_messages</code>	20 จำนวนเต็ม > 0 หรือ None	fsync เมื่อจำนวนข้อความ unsynced ถึงค่านี้ คำน้อยลด loss window แต่ sync บ่อยขึ้น
<code>sync_interval</code>	0.25 s > 0 หรือ None	fsync เมื่อมี dirty data ค้างถึงเวลา เหมาะสำหรับ stream อัตราต่ำ ที่ count trigger มาช้า
<code>sync_bytes</code>	65,536 64 KiB, จำนวนเต็ม > 0 หรือ None	fsync เมื่อ framed bytes ที่ยังไม่ sync ถึงค่า คุม batch ตามขนาดจริงมากกว่าจำนวน message
<code>ack_checkpoint_messages</code>	50 จำนวนเต็ม > 0 หรือ None	checkpoint cursor เมื่อ ACK สะสมถึงจำนวน คำน้อยลด replay window แต่เพิ่ม write/fsync
<code>ack_checkpoint_interval</code>	1.0 s > 0 หรือ None	checkpoint เมื่อมี ACK progress ค้างถึงเวลา ช่วย workload ที่ ACK มาไม่ถึง count threshold

กฎ validation: ปิด trigger รายตัวด้วย **None** ได้ แต่ต้องเหลือ data-sync trigger อย่างน้อย 1 ตัวจาก 3 ตัวแรก และ ACK-checkpoint trigger อย่างน้อย 1 ตัวจาก 2 ตัวหลัง มิฉะนั้น constructor จะ raise **ValueError**

ตัวอย่าง 3 รูปแบบ

```
# ค่าเริ่มต้น: เหมาะเป็นจุดเริ่มต้น
GroupMode()

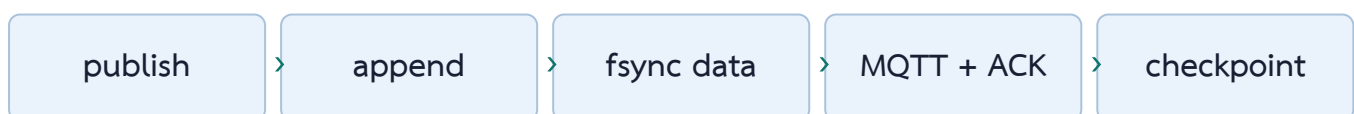
# เน้น count/bytes ไม่ใช่ timer ของ data
GroupMode(
    sync_messages=20,
    sync_interval=None,
    sync_bytes=64 * 1024,
    ack_checkpoint_messages=50,
    ack_checkpoint_interval=1.0,
)

# อัตราข้อมูลต่ำ: ใช้ timer เป็นหลัก
GroupMode(
    sync_messages=None,
    sync_interval=0.5,
    sync_bytes=None,
    ack_checkpoint_messages=None,
    ack_checkpoint_interval=1.0,
)
```

วิธีจูน: เริ่มจากค่า default วัด fsync latency, throughput และ replay ที่รับได้ แล้วปรับทีละกลุ่ม อย่าปิด timer หาก workload อาจหยุดก่อนถึง count/bytes โดยไม่เข้าใจผลของ loss window

1.4 DurableMode - strongest persistence

DurableMode ให้แต่ละข้อความผ่าน durable append ก่อน **publish()** คืบผล และให้ ACK progress ผ่าน durable checkpoint ก่อนลืมหักข้อความจาก persistent queue จึงเหมาะกับข้อมูลที่ต้องการสูญหายมีต้นทุนสูง



การตั้งค่า

DurableMode() ไม่มีพารามิเตอร์เฉพาะโหมด เพราะ semantics ตั้งใจให้ชัดเจนและปิด durability boundary ไม่ได้ หากต้องการ batch ให้เลือก GroupMode แทน

```
from reliomq import DurableMode, Sender

with Sender(config, mode=DurableMode()) as sender:
    message_id = sender.publish(
        "factory/critical/alarm",
        {"machine": "M-04", "active": True},
    )
    delivered = sender.wait_for_delivery(message_id, timeout=10.0)
```

Durable ไม่เท่ากับ exactly-once: การส่งยังเป็น at-least-once จึงอาจเกิด duplicate ในจังหวะ crash/เครือข่ายผิดปกติ Consumer ควร deduplicate ด้วย message ID

1.5 Recommended mode selection

คำถาม	คำตอบ	Mode
หากไฟดับทันที ข้อมูลใหม่ล่าสุดหายได้หรือไม่?	ได้ และอยากลดการเขียน SD มากที่สุด	FastMode
ต้อง append ทุกข้อความ แต่ยอมรับ loss/replay window สั้น ๆ ได้หรือไม่?	ได้ ต้องการสมดุล	GroupMode
ข้อความที่ publish รับแล้วควรรอดจาก crash ให้มากที่สุดหรือไม่?	ใช่	DurableMode

ถ้าไม่แน่ใจ: ใช้ GroupMode default สำหรับ telemetry ที่ต้อง persist หรือ DurableMode สำหรับเหตุการณ์สำคัญ อย่าใช้ FastMode เพียงเพราะเป็นค่า default หากข้อมูล RAM-only หายไม่ได้

2. Deep Technical Detail

2.1 Storage architecture

Outbox เป็น persistent FIFO แบบแบ่ง segment Payload อยู่ในไฟล์ segment ที่เรียงหมายเลข ส่วน logical head อยู่ใน cursor ที่ระบุ `segment_id` และ `byte_offset` การ ACK จึงเลื่อนหัวคิวแทนการ rewrite ข้อความที่เหลือทั้งหมด

```
outbox/
|-- format.json
|-- checkpoint.json
|-- segment-00000000000000000001.dat
|-- segment-00000000000000000002.dat
`-- ...
```

```
{
  "version": 1,
  "generation": 37,
  "segment_id": 2,
  "byte_offset": 48291
}
```

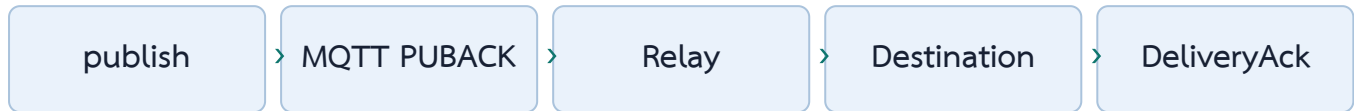
ความหมายคือ record ก่อน cursor เสร็จงานเชิงตรรกะแล้ว ส่วน record ตั้งแต่ cursor เป็นต้นไปยัง pending

2.2 What write, append, flush, and fsync mean

Operation	ความหมาย	ผลด้าน durability
append/write	ส่ง bytes เข้าสู่เส้นทางเขียนของ OS ซึ่งมักเริ่มที่ page cache	เร็ว แต่ยังไม่รับประกันว่าอยู่บน stable storage
userspace flush	ผลัก buffer ของ runtime/library ลงไปยัง OS	ยังไม่เท่ากับ fsync
fsync	ขอให้ OS ทำ data/metadata ที่จำเป็นให้ durable และรอผล	เป็น explicit durability boundary ของไลบรารี
checkpoint	เขียน cursor ปัจจุบันและ sync state ที่เกี่ยวข้อง	ทำให้ ACK progress กู้คืนได้และเปิดทางลบ segment ที่ใช้หมด

2.3 ACK model: DeliveryAck versus MQTT transport ACK

MQTT PUBACK ยืนยันการส่ง QoS 1 ใน hop หนึ่ง ส่วน **DeliveryAck** ของ reliomq ยืนยัน workflow ระดับ application ว่า Relay ส่งไป destination broker แล้ว Sender จึง complete record ที่เก่าสุดซึ่งตรงกับ ACK



`mqtt_puback_timeout` และ `delivery_ack_timeout` เป็น timeout คนละชั้น ทั้งคู่ทำงานใน background worker และไม่ใช่ timeout ทั้ง workflow ของ `wait_for_delivery()`

2.4 checkpoint.json and ACK persistence

ACK progress ถูกแทนด้วย queue cursor ไม่ใช่ ACK journal ที่เพิ่มทีละบรรทัด ค่า `generation` เพิ่มเมื่อ checkpoint ใหม่ติดตั้งสำเร็จ

```

ก่อน ACK:  [msg1][msg2][msg3][msg4]
             ^ cursor

ACK msg1:   [msg1][msg2][msg3][msg4]
             ^ volatile cursor

checkpoint:  serialize cursor -> write -> fsync -> atomic install
  
```

`fsync` เพียงอย่างเดียวไม่สามารถเก็บ cursor ค่าใหม่ที่ไม่เคย serialize และ write ลงไฟล์ ดังนั้นลำดับการ checkpoint จึงสำคัญ

2.5 FastMode in detail

2.5.1 Healthy path

```

t0 publish(msg1)    RAM: [msg1]    Disk: unchanged
t1 send(msg1)       RAM: [ ]       Disk: unchanged
t2 DeliveryAck      RAM: [ ]       Disk: unchanged
  
```

เส้นทางปกติไม่มี payload append, ไม่มี payload fsync และไม่มี ACK checkpoint สำหรับข้อความ RAM-only

2.5.2 Spill path

เมื่อ trigger ทำงาน FastMode เลือก record เก่าสุดตาม batch limit แล้ว append เป็นชุดที่มี durable sync boundary โดยเก็บ RAM copy ไว้จนเขียนสำเร็จ เพื่อกันข้อมูลหายเมื่อ disk ล้มเหลว

```
RAM [msg10][msg11][msg12] -> append_many -> segment -> data fsync
```

หลัง restart: โหลด disk-backed records -> resend -> DeliveryAck -> durable checkpoint

2.5.3 Power-loss consequence

หากไฟดับก่อนข้อความ RAM-only ถูก spill ข้อความนั้นอาจหาย เมื่อ spill และ sync สำเร็จแล้ว ข้อความจะใช้ recovery path ของ Outbox การตั้ง `disconnect_grace=None` จึงลด disk write ตอน offline แต่เพิ่มเวลาที่ pending data อยู่เฉพาะ RAM

2.6 GroupMode in detail

2.6.1 Payload append behavior

ทุก publish append ด้วย sync disabled แล้วเพิ่มตัวนับ unsynced messages/bytes หาก count, bytes หรือ interval ตัวใดถึงก่อน จะเรียก Outbox data sync

```
ตัวอย่าง 5 msg/s และ sync_interval=0.25
t=0.00 append msg1 dirty
t=0.20 append msg2 dirty
t=0.25 timer fires -> Outbox.sync() -> fsync segment data
```

ที่ 5 msg/s ค่า default มักโดน timer ก่อน `sync_messages=20` ดังนั้นการคำนวณ sync rate จาก batch count อย่างเดียวอาจผิด

2.6.2 ACK behavior

เมื่อ DeliveryAck ถูกต้อง GroupMode เลื่อน volatile cursor ใน RAM ก่อน แล้วเพิ่ม ACK count เมื่อ count/interval ถึง หรือ closed segment ถูกใช้หมด จะ checkpoint cursor โดย sync data ที่จำเป็นก่อน

```
ACK1 + ACK2 + ACK3 -> RAM cursor advances
                        |
                        \-> checkpoint trigger
                            1. sync required data
                            2. write checkpoint
                            3. fsync / install
                            4. reclaim closed segment when safe
```

2.6.3 Why replay can occur after power loss

ถ้า msg1 ถึง msg3 ถูก ACK แล้ว แต่ disk checkpoint ยังชี้ที่ msg1 เมื่อไฟดับ RAM cursor จะหาย หลัง restart จึงอ่าน checkpoint เก่าและส่ง msg1 ถึง msg3 ซ้ำ นี่คือพฤติกรรม at-least-once ที่เลือก replay แทนการลิ้มข้อความโดยไม่มีหลักฐาน durable ว่าเสร็จแล้ว

ประมาณ loss window: ไม่มีสูตรตายตัวเพราะ trigger ใดถึงก่อนก็ได้ ใช้ค่าที่น้อยที่สุดในเชิงเวลาโดยพิจารณา rate และขนาด message เช่น count/rate, bytes/byte-rate และ interval แล้วเพื่อ fsync latency

2.7 DurableMode in detail

2.7.1 Payload durability

```
publish(msg1) -> append to segment -> fsync segment -> publish() returns
```

หาก durable append ล้มเหลว การรับข้อความไม่ควรถูกตีความว่าสำเร็จ ผู้เรียกต้องจัดการ exception และตรวจ storage

2.7.2 ACK durability

```
DeliveryAck -> calculate next cursor -> sync required data
               -> write checkpoint.json -> fsync / atomic install
               -> remove live pending record -> safe segment cleanup
```

ลำดับนี้ลดช่วงที่ ACK แล้วแต่ disk ยังไม่รู้ให้น้อยที่สุด แต่ duplicate ยังเป็นไปได้ในระบบ distributed จึงต้องออกแบบ consumer ให้ idempotent

2.8 What the files may look like

Mode	Segment files	checkpoint.json	รูปแบบ update
Fast	อาจไม่เปลี่ยนบน healthy path; ใช้เมื่อ spill/recovery	เกี่ยวข้องกับ disk-backed records	ไม่มี per-message disk write บน RAM-only path
Group	append ทุกข้อความ, fsync เป็นชุด	cursor schema เดียวกัน	หลาย ACK รวมใน checkpoint เดียวได้
Durable	append + sync ทุก persistent message	cursor schema เดียวกัน	completion checkpoint ต่อ disk-backed ACK

ทั้ง 3 โหมดไม่ได้ใช้ ACK file format คนละแบบ ความต่างอยู่ที่ว่าข้อความลง disk เมื่อใด และ sync/checkpoint บ่อยแค่ไหน

2.9 Crash and restart scenarios

Scenario	FastMode	GroupMode	DurableMode
ไฟดับก่อน payload durable	RAM-only อาจหาย	append ล่าสุดที่ยังไม่ sync อาจหาย	publish ที่คืนสำเร็จควรผ่าน sync boundary แล้ว

Scenario	FastMode	GroupMode	DurableMode
ส่งถึงแล้วแต่ ACK state ยังไม่ durable	disk-backed completion ใช้ durable path	อาจ replay ตาม checkpoint เก่า	checkpoint ต่อ completion ลด window
clean shutdown	spill pending ตาม shutdown logic	flush data และ ACK checkpoint ที่ค้าง	แต่ละ operation durable อยู่แล้ว
process ถูก kill/power cut	อย่าคาดหวัง clean-shutdown spill	อย่าคาดหวัง deferred timer ทำงานทัน	ขอบเขตที่สำเร็จก่อน crash ยังมีหลักฐาน disk

2.10 Storage-wear interpretation

จำนวน Python append ไม่เท่ากับจำนวน flash program operation เพราะ OS และ controller อาจ cache/รวม write การเปรียบเทียบที่มีความหมายกว่าคือความถี่ของ forced synchronization และปริมาณข้อมูล/metadata ที่ต้องเขียน

ที่ 5 msg/s (ตัวอย่าง)	พฤติกรรมโดยประมาณ
Fast healthy	RAM -> ACK; persistent payload fsync ปกติเป็น 0
Group default	5 appends/s; data timer สูงสุดราวทุก 0.25 s เมื่อมี dirty data; ACK checkpoint สูงสุดราวทุก 1.0 s หรือ 50 ACK
Durable	ทุก persistent message และ completion ผ่าน sync/checkpoint boundary

ตัวเลขจริงขึ้นกับจังหวะ ACK, segment rotation, filesystem, storage และ latency ของ fsync จึงควรวัดบน hardware เป้าหมาย

2.11 Why GroupMode has two independent durability windows

data sync ป้องกัน payload ใหม่ ส่วน ACK checkpoint ป้องกันความรู้ว่าข้อความเก่าส่งเสร็จแล้ว หากบังคับใช้ค่าเดียวจะจน loss และ duplicate replay แยกกันไม่ได้ GroupMode จึงมี trigger สองกลุ่มอิสระ

Payload-loss window

sync_messages
sync_interval
sync_bytes

Duplicate-replay window

ack_checkpoint_messages
ack_checkpoint_interval

ACK checkpoint จะ sync deferred data ที่จำเป็นเพื่อ recovery ด้วย จึงอาจจบ data window ก่อน data trigger ของตัวเองถึงกำหนด

2.12 Practical explanation for end users

- **FastMode:** เก็บ traffic ปกติใน RAM เพื่อความเร็วและลด storage wear ลง disk เมื่อเกิด spill เท่านั้น ข้อมูล RAM-only อาจหายเมื่อไฟดับ
- **GroupMode:** append ทุกข้อความ แต่ batch ทั้ง fsync และ ACK checkpoint ให้สมดุลขึ้น ไฟดับอาจทำให้ payload ล่าสุดท้ายหรือข้อความที่ ACK แล้วถูกส่งซ้ำ
- **DurableMode:** ทำ durability boundary ต่อข้อความและ completion ให้การป้องกัน crash สูงสุด แลกกับ latency และ storage activity สูงสุด

2.13 Glossary

คำ	ความหมาย
Append	เพิ่ม record ต่อท้าย active segment
Dirty data	ข้อมูลที่แก้ไขแล้วแต่อาจยังอยู่ใน memory/page cache และยังไม่จบ writeback
fsync	system call ที่บังคับผลัก data/metadata ที่จำเป็นไป storage และรอผล
Outbox	persistent FIFO สำหรับ disk-backed message ที่ยังไม่ complete แบบ durable
Segment	ไฟล์ data ที่มี framed message records และมีหมายเลขเรียงลำดับ
Cursor	ตำแหน่ง <code>(segment_id, byte_offset)</code> ที่แทน logical head ของ FIFO
Checkpoint	ตัวแทน cursor แบบ durable ใน <code>checkpoint.json</code>
DeliveryAck	application-level acknowledgement ที่ complete ข้อความเก่าสุดที่ตรงกัน
Replay	ส่งข้อความซ้ำหลัง restart เพราะ completion ยังไม่ถูก checkpoint
At-least-once	เน้นไม่ลืมข้อความ จึงยอมให้ duplicate เกิดและควรใช้ idempotent consumer

ข้อแนะนำสุดท้าย: วัดบนอุปกรณ์จริงด้วยอัตราข้อมูลจริง ทดสอบ power loss และทำ deduplication ที่ consumer ไม่ว่าเลือกโหมดใด

แหล่งอ้างอิง implementation: `reliomq/durability.py`, `reliomq/persistence.py`, `reliomq/outbox.py` บน main ของ reliomq 0.6.2